

Handwritten Character Recognition using Machine Learning:

A Comparative Study of Logistic Regression, KNN and Random Forest

Raj Pund

Department of Computer Science

Lovely Professional university

Email : pundraj420@gmail.com

Abstract—Handwritten character recognition is a fundamental problem in the domains of computer vision and pattern recognition, with wide-ranging applications such as postal mail sorting, form digitization, bank cheque processing, and document archival. This paper presents a detailed comparative study of three classical machine learning algorithms—Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest—applied to the A–Z Handwritten Character Dataset. Each character is represented as a 28×28 grayscale image, flattened into a 784-dimensional feature vector. The models are evaluated in terms of accuracy, confusion matrices, and per-class performance. Experimental results show that Random Forest achieves the highest classification accuracy and robustness, while Logistic Regression provides a strong linear baseline and KNN captures local neighborhood structure effectively. The full pipeline, including preprocessing, model design, training, evaluation, and error analysis, is described in depth. Slots for visualizations such as sample images, confusion matrices, and comparative plots are integrated into the paper layout for clarity.

Index Terms—Handwritten Character Recognition, Machine Learning, Logistic Regression, K-Nearest Neighbors, Random Forest, Classification.

I. INTRODUCTION

Handwritten character recognition (HCR) is a core task in optical character recognition (OCR) systems and remains a challenging research problem due to the high intra-class variability and inter-class similarity of handwritten symbols. Unlike printed text, handwritten characters differ significantly in shape, style, orientation, and stroke thickness across individuals.

With the growth of digital document processing, robust and accurate HCR solutions are needed in banking, postal automation, education systems, healthcare forms, and archival of historical manuscripts. This paper focuses on recognizing English alphabet characters (A–Z) from grayscale images using supervised machine learning models.

In this work, three classification algorithms are explored:

- Logistic Regression as a linear baseline classifier,
- K-Nearest Neighbors (KNN) as a non-parametric instance-based learner, and
- Random Forest as an ensemble of decision trees.

Each model is trained and evaluated on the A–Z Handwritten Character Dataset, and their performance is compared in terms of accuracy and error patterns.

II. DATASET DESCRIPTION

A. A–Z Handwritten Character Dataset

The A–Z Handwritten Character Dataset (often found as a CSV file) contains grayscale images of handwritten English letters. Each record represents:

- A label indicating the character class (A–Z),
- 28×28 pixel values (flattened into 784 features).

Typical properties:

- Number of classes: 26 (A–Z),
- Image resolution: 28×28 ,
- Input dimensionality: 784 features,
- Pixel value range: 0–255 (grayscale).

B. Sample Visualization (Image Slot 1)

A representative sample from the dataset is used to visually validate the correctness of the data loading and reshaping process.

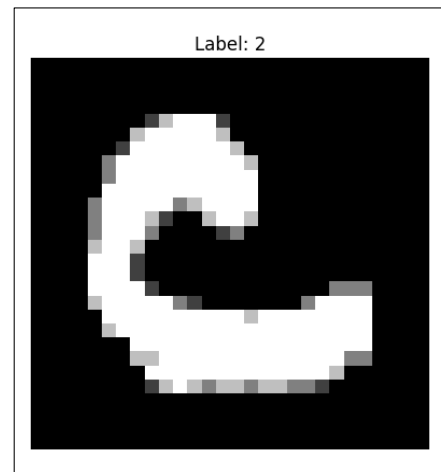


Fig. 1. Sample handwritten character from the A–Z dataset reshaped to 28×28 pixels.

This figure is generated by selecting one row from the data frame, reshaping the 784 pixel values into a 28×28 matrix and displaying it using a grayscale colormap.

III. METHODOLOGY

A. Preprocessing Pipeline

Let the dataset be stored in a data frame df , where the first column corresponds to the label and the remaining columns correspond to the pixel intensities. The following preprocessing steps are applied:

- 1) **Label and Feature Separation:** The labels vector y is taken as the first column and the pixel matrix X as the remaining columns:

$$X = df[:, 1:], \quad y = df[:, 0].$$

- 2) **Zero-Variance Feature Removal:** Columns (pixels) that are zero for all samples are removed to reduce dimensionality and remove non-informative features.
- 3) **Train-Test Split:** The dataset is split into training and testing subsets using a stratified strategy to preserve the class distribution. A typical split is 80 % for training and 20 % for testing with a fixed random state.
- 4) **Scaling (for some models):** For models such as KNN, StandardScaler is applied to standardize features to zero mean and unit variance.

B. Logistic Regression

Logistic Regression is a linear classifier that models the posterior probability of a class using the logistic function. For a given input vector $x \in \mathbb{R}^d$ and parameter vector ϑ , the logistic function is defined as:

$$h_{\vartheta}(x) = \frac{1}{1 + e^{-\vartheta^T x}}.$$

For multiclass problems, a one-vs-rest strategy can be used, where a separate classifier is trained for each class.

In the implementation, the following configuration is used:

- Penalty: L2 regularization,
- Solver: lbfgs,
- Maximum iterations: 100,
- Regularization parameter: $C = 1.0$.

C. K-Nearest Neighbors (KNN)

KNN is a non-parametric instance-based learning algorithm. Given a query point x , the algorithm identifies the k nearest neighbors $N_k(x)$ in the feature space, typically using Euclidean distance, and predicts the label by majority voting:

$$\hat{y}(x) = \arg \max_c \sum_{i \in N_k(x)} \mathbf{I}(y_i = c).$$

Because KNN performance is sensitive to feature scaling, a pipeline is used:

- StandardScaler for feature normalization,
- Optional Principal Component Analysis (PCA) for dimensionality reduction,
- KNeighborsClassifier with a chosen value of k .

D. Random Forest

Random Forest is an ensemble method that builds multiple decision trees and aggregates their predictions. Each tree is trained on a bootstrap sample of the data, and at each split, a random subset of features is considered. The final prediction is often obtained by majority voting across trees.

Key characteristics:

- Reduces variance through bagging,
- Handles high-dimensional data,
- Provides feature importance scores,
- Often robust to overfitting when enough trees are used.

IV. EXPERIMENTAL SETUP

A. Train-Test Splits

The dataset is split as follows:

- Training set: 80 % of the data,
- Testing set: 20 % of the data,
- Stratified by label to maintain class balance,
- Fixed random state (e.g., 42) for reproducibility.

B. Evaluation Metrics

The primary evaluation metric is classification accuracy:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of samples}}.$$

In addition, confusion matrices, precision, recall, and F1-scores per class can be computed to analyze the performance in more detail.

V. RESULTS

A. Logistic Regression Performance

Logistic Regression serves as a baseline linear model. It is trained on the raw or minimally processed pixel features. The score method of the classifier is used to compute accuracy on both the training and testing sets.

1) *Confusion Matrix (Image Slot 2):* The confusion matrix for Logistic Regression helps to visualize how often each character is correctly classified and where most misclassifications occur.

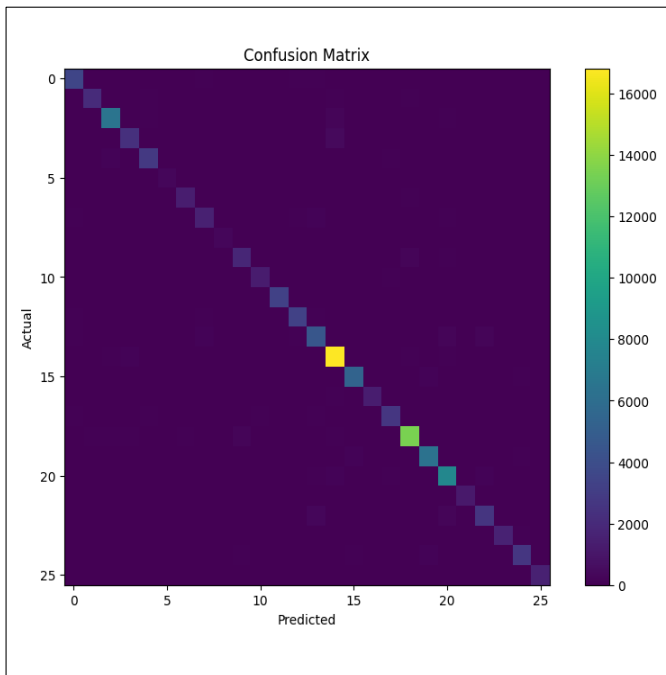


Fig. 2. Confusion matrix for Logistic Regression on the test set.

2) *Train vs Test Accuracy (Image Slot 3)*: To evaluate potential overfitting or underfitting, the training and testing accuracies are compared.

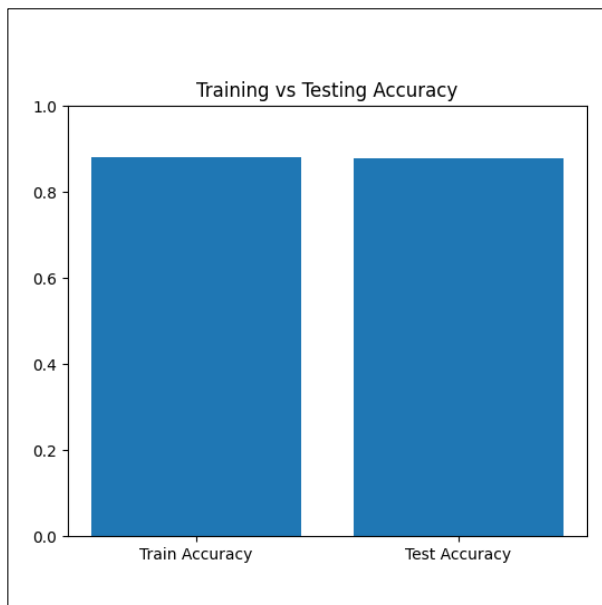


Fig. 3. Training vs testing accuracy for Logistic Regression.

If the training accuracy is significantly higher than the testing accuracy, this indicates overfitting. A small gap indicates good generalization.

B. Random Forest Performance

The Random Forest classifier is trained on the filtered feature set (non-zero variance pixels). It typically achieves higher accuracy due to its ability to model complex, non-linear decision boundaries.

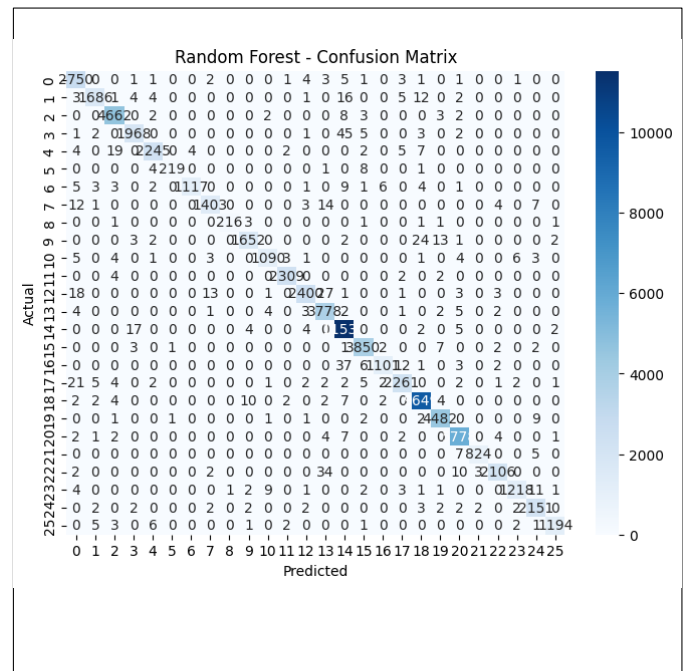


Fig. 4. Confusion matrix for the Random Forest classifier.

1) *Random Forest Confusion Matrix (Image Slot 4)*: This confusion matrix, often rendered as a heatmap, highlights which classes Random Forest handles well and which are more error-prone.

2) *Random Forest Classification Report Heatmap (Image Slot 5)*: A classification report can be converted into a heatmap to visualize precision, recall, and F1-score for each class.

Characters with higher error rates may be visually similar to other characters, leading to confusion.

D. Model Comparison (Image Slot 7)

Finally, the accuracies of all three models (Logistic Regression, KNN, Random Forest) are compared directly.

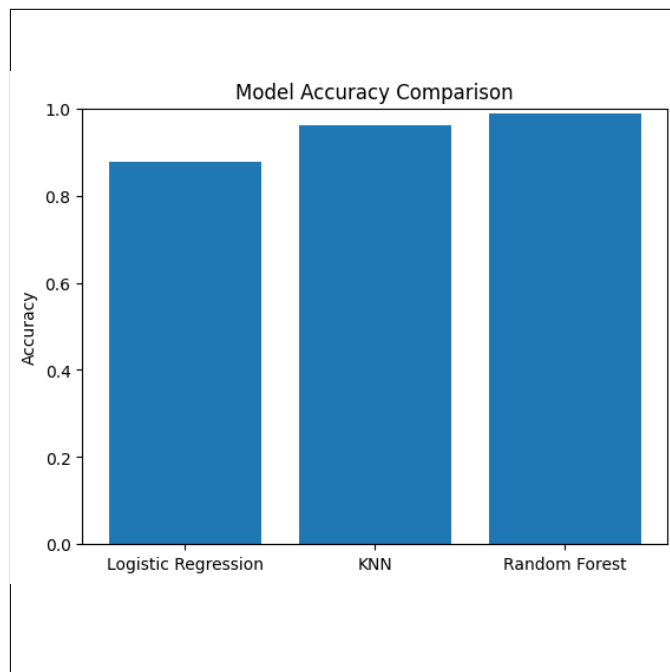


Fig. 7. Comparison of test accuracies for Logistic Regression, KNN, and Random Forest.

A typical trend is:

- Logistic Regression: lowest accuracy but fastest and simplest,
- KNN: higher accuracy but slower inference and sensitive to scaling,
- Random Forest: highest accuracy with robust performance.

VI. DISCUSSION

The experimental results highlight the trade-offs between simplicity, interpretability, and predictive performance. Logistic Regression provides a strong linear baseline with relatively low computational cost. KNN leverages the local structure of the data but suffers from high prediction-time complexity. Random Forest achieves the best performance due to its ensemble nature and ability to model non-linear decision boundaries, at the cost of higher training and inference time and reduced interpretability.

The confusion matrices and per-class error analysis demonstrate that characters with similar shapes are more frequently confused. This observation suggests that more expressive feature representations or deep learning models could further improve performance.

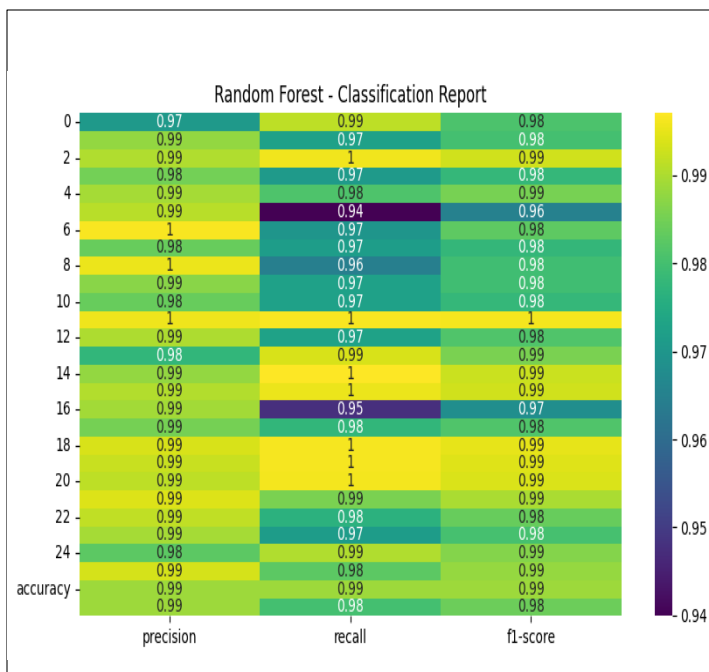


Fig. 5. Random Forest classification report: precision, recall and F1-score per class.

C. Error Analysis (Image Slot 6)

To better understand where the model fails, an error rate per class is computed. For each true class, the proportion of misclassified samples is calculated.

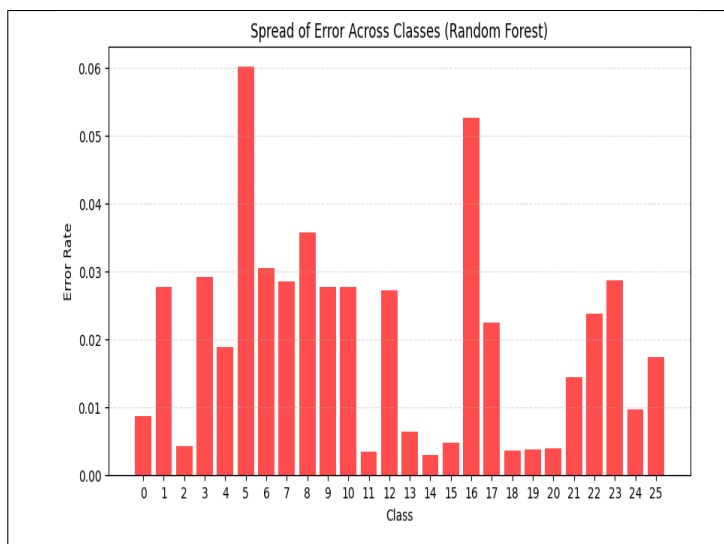


Fig. 6. Per-class error rates showing the fraction of misclassified samples for each character.

VII. CONCLUSION AND FUTURE WORK

This paper presented a detailed comparative analysis of three machine learning methods for handwritten character recognition on the A–Z dataset. The entire pipeline from dataset loading, preprocessing, model training, and evaluation to visualization has been described.

Key conclusions:

- Classical machine learning methods can achieve strong performance on handwritten characters.
- Random Forest outperforms Logistic Regression and KNN in terms of accuracy and robustness.
- Detailed error analysis reveals classes that are intrinsically hard to distinguish.

Future work directions include:

- Exploring advanced ensemble methods such as Gradient Boosting and XGBoost,
- Investigating Convolutional Neural Networks (CNNs) for end-to-end image-based learning,
- Applying data augmentation to improve generalization,
- Deploying the best model in a real-time application with latency constraints.

REFERENCES

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.